Programming Project 2

Replicated Object Store

Paul Krzyzanowski

2026-03-06

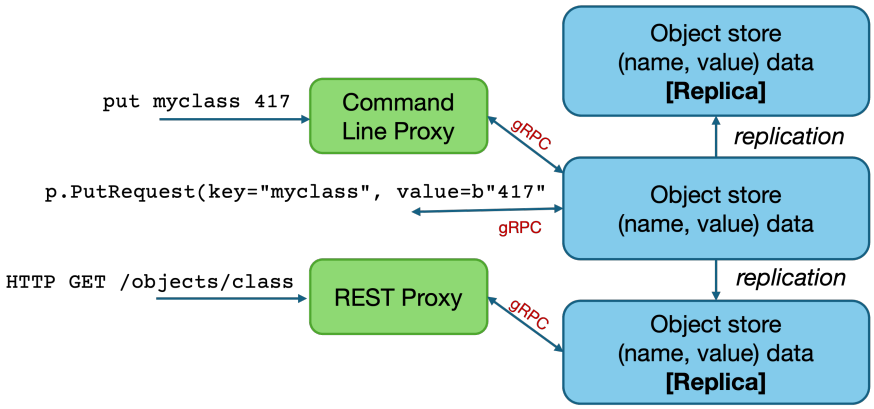
Overview

In this project, you will build a distributed object store with three components:

1. a replicated storage server (one or more instances of the server program),
2. a REST-based web proxy, and
3. a command-line interface (CLI) proxy.

The storage service exposes a gRPC interface defined by a proto file we supply. An external program can interact with this directly.

Both proxies translate their respective front-end protocols into gRPC calls to the back end, enabling different interface options to the service.



The goal is to give you hands-on experience with RPC design, serialization, protocol layering, distributed replication, and realistic benchmarking.

You will implement the project in Python. We provide `objectstore.proto`; you generate the Python stubs from it using `grpcio-tools`.

The Object Store

A memory-based key-value store is a common building block in distributed systems. It shows up as a cache for web services (to reduce latency and load), a coordination service for small pieces of shared state, a metadata store for names, IDs, and configuration blobs, and a fast staging area in front of a slower persistent store.

Even when the underlying idea is basic (map a key to a value), real systems quickly become complicated when you add concurrency, RPC boundaries, replication, partial failures, and

performance measurement. This project is designed to make you implement and test some of these details directly.

We'll ignore cases of replicas restarting, failover to a replica, and adding a new replica. All those would be critical for a production system, but would make the assignment more complex. I'd encourage you to try adding those after you're done to see what it takes to build a production-grade service.

High-Level Goals

You will implement three programs that work together.

The gRPC server is the core of the system. The other two programs are front ends that make the service usable from different environments. This is a common pattern in real systems: one backend service with multiple client-facing interfaces.

The Programs

1. Object Store Server (`server.py`)

This is the main program. It stores `(key, value)` pairs in memory and exports a gRPC interface defined by `objectstore.proto`. The server can run as a single node or as a small cluster of N processes. In cluster mode, one process is designated as the **primary** and the rest are **replicas**.

The primary is the only node that accepts write operations from clients. When the primary processes a write, it propagates the update to all replicas via gRPC so they can serve future reads. Reads may go to any node.

The server must:

- Store `(key, value)` pairs in memory and enforce key and value limits.
- Implement the gRPC interface exactly as defined in `objectstore.proto`.
- Operate correctly as a single node when the cluster list contains only one entry.
- Operate in a primary-and-replica configuration when the cluster list contains more than one entry, with the primary propagating writes to replicas via `ApplyWrite`.
- Return correct gRPC status codes for all error conditions.
- Implement `Stats` to report operational counters.

2. REST Proxy (`restproxy.py`)

This program provides an alternate front end to the object store by exposing a RESTful HTTP API.

Many environments can call HTTP endpoints more easily than gRPC. A web page running JavaScript, a shell script using `curl`, or a monitoring dashboard can all speak HTTP without needing a gRPC client library. The REST proxy translates incoming HTTP requests into gRPC calls to the backend cluster and translates gRPC status codes into HTTP status codes.

To help you get started, we provide a working skeleton (`restproxy.py`) and a separate implementation guide (`restproxy-guide.md`). The skeleton handles routing, argument parsing, gRPC stub setup, and status-code translation. You fill in the individual handler methods. Read the guide before you start.

The REST proxy must:

- Listen for HTTP requests and implement the required REST endpoints.
- Forward write requests to the primary and read requests to any node.
- Translate gRPC status codes into appropriate HTTP status codes.
- Preserve values as opaque bytes without encoding or transforming them.

3. CLI Proxy (`cli.py`)

This program lets you interact with the object store by typing commands at a terminal. It reads one command per line (for example, `put mykey somevalue` or `get mykey`) and prints results and errors as text. This kind of interface is useful for manual testing, demos, and simple automation.

Like the REST proxy, the CLI does not store data. It communicates with the backend cluster using gRPC.

The CLI proxy must:

- Accept commands on standard input and print responses to standard output.
- Forward write commands to the primary and read commands to any node.

What You Are Given

- `objectstore.proto`: defines the gRPC service, all message types, and RPC signatures. Do not modify it.
- `restproxy.py`: a working skeleton for the REST proxy with routing and infrastructure already implemented.
- `restproxy-guide.md`: a detailed explanation of the skeleton and instructions for filling it in. Read this before editing `restproxy.py`.

You will generate Python gRPC stubs from the proto file:

```
python3 -m grpc_tools.protoc \
    -I. \
    --python_out=. \
    --grpc_python_out=. \
    objectstore.proto
```

This produces `objectstore_pb2.py` (message classes) and `objectstore_pb2_grpc.py` (client stubs and server base classes). Import both in your server and proxy files.

Your implementation must match the protobuf definitions and the behavioral requirements in this specification exactly.

Incremental Development Guide

This project has several moving parts. The worst thing you can do is try to build all of them at once and then debug a system where everything is broken simultaneously. The following sequence will save you significant time and/or sanity.

Stage 1: Get gRPC working end to end

Before you write any real logic, verify that gRPC communication works between two Python processes on your development machine. Write a minimal server that registers the `ObjectStore` service, receives each RPC, prints the request to the terminal, and returns a canned response: a hardcoded value for `Get`, an empty response for `Put`, and so on. Write a throwaway client script that calls each RPC in sequence and prints the result.

This code will not be submitted. Its only purpose is to confirm that proto compilation, stub imports, channel setup, and basic message serialization all work before you invest time in real logic. If this stage is broken, nothing else will work. Get it solid first.

Stage 2: Implement the single-node server

Now implement real server logic: the in-memory store, all seven client-facing RPCs, key and value validation, correct gRPC status codes, and the `Stats` counters. Run it as a single node:

```
python3 server.py --listen localhost:50051 --cluster localhost:50051
```

Test it using your throwaway client from Stage 1, extended to exercise every operation and every error path: `put` followed by `get`, `put` on an existing key (expect `ALREADY_EXISTS`), `get` on a missing key (expect `NOT_FOUND`), `list` before and after modifications, `stats` counter accuracy, and `reset`.

Do not proceed until every operation is correct on a single node. Bugs here will compound badly once you add replication.

Stage 3: Implement the CLI proxy

Implement `cli.py` next. Once it works, you can test the server interactively with readable commands rather than maintaining a throwaway client script. Write a `testcmds.txt` file with a sequence of commands covering every operation and run it against your server:

```
python3 cli.py --cluster localhost:50051 < testcmds.txt
```

If the output is not what you expect, the bug is in the server or the CLI, and at this stage it is easy to tell which. The CLI is also the tool you will use to debug replication in Stage 4, so having it working first is worth the effort.

Stage 4: Add replication

Note that all your initial testing, for simplicity and sanity, should be done on one machine.

Now should you add replication. Extend the server to send writes to replicas via `ApplyWrite`, implement the majority-commit logic, and implement write rejection on replicas.

Start a two-node cluster on your local machine using different ports:

```
python3 server.py --listen localhost:50051 --cluster localhost:50051,localhost
:50052 &
python3 server.py --listen localhost:50052 --cluster localhost:50051,localhost
:50052 &
```

Note in this example that we start both instances of the server on the same machine (identified by `localhost`) with each instance listening on different ports.

Note also that this example runs the servers in the background (the `&` at the end of each command). You might want to run each command in a separate window instead. That way, any debugging output will not be interleaved, and it will be clear which process is generating the output. You also won't run the risk of forgetting about background processes that are still running when you try to start new versions at a later time.

Use `cli.py` to send writes and verify that both nodes reflect the update by querying each directly. Kill one replica and confirm that writes still succeed. Then point the CLI at the replica and try a write. You should receive `FAILED_PRECONDITION`.

Stage 5: Implement the REST proxy

Implement `restproxy.py` using the provided skeleton. By this point the server is fully correct and you have a clear mental model of each RPC. Use the `curl` command to test each endpoint as you fill in the handler methods. See the testing examples in `restproxy-guide.md`.

Stage 6: Benchmarks and write-up

With all three programs working correctly, run the two benchmarks on iLab machines (any other system should work for debugging, but you should ultimately ensure they run in an iLab environment). Run your benchmark scripts locally and test them against a local cluster before moving to multiple machines. It is much faster to iterate on scripts locally.

Formal Specification

Key Rules

The key is a human-friendly piece of text that gives the object (value) a name. A key is an ASCII string with the following constraints:

- Characters must be printable ASCII (0x21 through 0x7E, inclusive). No spaces, tabs, or control characters are permitted.
- Maximum length: 128 characters.
- Keys are case-sensitive: `Photo` and `photo` are different keys.

Value Rules

A value is the object associated with a key. A value is an arbitrary sequence of bytes. Clients treat values as opaque: the server does not inspect or transform their content.

- Maximum size: 1 MiB (1,048,576 bytes).
- Values are typed `bytes` in the proto definition, not `string`. This allows, for example, storing a JPEG file without encoding it as text.

gRPC Interface

The full interface is defined in `objectstore.proto`, which we supply. Do not modify it. The service definition is:

```
Put    (PutRequest)            -> Empty
Get    (GetRequest)            -> GetResponse
Delete (DeleteRequest)         -> Empty
Update (UpdateRequest)         -> Empty
List   (Empty)                 -> ListResponse
Reset  (Empty)                 -> Empty
Stats  (Empty)                 -> StatsResponse
ApplyWrite (WriteOp)           -> Empty    # primary -> replicas only
```

`List` returns a `ListResponse` containing a repeated `ListEntry` field. Each entry has the key and the size of the value in bytes.

`ApplyWrite` is used only for replication between the primary and its replicas. Clients must never call it directly.

Operation Semantics

put(key, value)
Stores the key-value pair.

- Returns `ALREADY_EXISTS` if the key already exists.
- Returns `INVALID_ARGUMENT` if the key violates the key rules or the value exceeds 1 MiB.
- Returns `OK` on success.

get(key)
Returns the value associated with the key.

- Returns `NOT_FOUND` if the key does not exist.
- Returns `OK` with the value on success.

delete(key)
Deletes the object.

- Returns `NOT_FOUND` if the key does not exist.
- Returns `OK` on success.

update(key, value)
Replaces the value of an existing object.

- Returns `NOT_FOUND` if the key does not exist.
- Returns `INVALID_ARGUMENT` if the new value exceeds 1 MiB.
- Returns `OK` on success.

list()
Returns all stored objects in a single `ListResponse`. Each entry contains the key and the size of the value in bytes. The response must be consistent: it must not reflect a partially applied write.

reset()
Deletes all objects and resets all operation counters to zero. Always returns `OK`.

stats()
Returns a snapshot of server-side counters. `live_objects` is the current number of stored keys. `total_bytes` is the sum of value sizes across all live objects. The remaining counters track successful operations since startup. A failed `put` does not increment the put counter.

Atomicity

Each RPC must be atomic with respect to the key it touches. A concurrent `put` and `delete` on the same key must produce a valid outcome: one succeeds and one returns an appropriate status code. Document the synchronization primitives you use and explain why you chose them.

The `list` RPC must not crash or return corrupted data under concurrent modifications, but it is not required to be serializable with respect to concurrent writes.

Distribution Model

Cluster Membership and Primary Selection

Every server program is started with two flags:

```
--listen host:port
    The local address this server binds to.

--cluster host1:port1,host2:port2,...
    The full membership list. Every node in the cluster gets the same list.
```

At startup, each server:

1. Parses `--cluster` by splitting on commas and trimming whitespace from each entry.
2. Validates that every entry is a well-formed `host:port`. Fail fast with a clear error message if not.
3. Requires that `--listen` appears in the parsed cluster list. Fail fast if it does not.
4. Normalizes each endpoint with `.lower().strip()` and defines the **primary** as the lexicographically smallest endpoint in the normalized list.

Because every instance of the service applies the same deterministic rule to the same list, they agree on the primary without any additional coordination. There is no configuration file and no bootstrap step.

Single-node mode. If the cluster list contains exactly one entry, the server is both primary and the entire cluster. It applies writes locally, skips the `ApplyWrite` fan-out step entirely, and trivially satisfies majority (1 out of 1). No special flag is needed; the replication logic handles this naturally when the replica list is empty. Single-node mode is how you develop and test the server in Stages 1 through 3, and it is the baseline configuration for Benchmark 2.

Here's an example of starting a 3-node cluster across three iLab machines. Run one command per machine:

```
python3 server.py --listen ilab1.cs.rutgers.edu:50051 \
  --cluster ilab1.cs.rutgers.edu:50051,ilab2.cs.rutgers.edu:50051,ilab3.cs.rutgers.edu:50051

python3 server.py --listen ilab2.cs.rutgers.edu:50051 \
  --cluster ilab1.cs.rutgers.edu:50051,ilab2.cs.rutgers.edu:50051,ilab3.cs.rutgers.edu:50051

python3 server.py --listen ilab3.cs.rutgers.edu:50051 \
  --cluster ilab1.cs.rutgers.edu:50051,ilab2.cs.rutgers.edu:50051,ilab3.cs.rutgers.edu:50051
```

Here, `ilab1.cs.rutgers.edu:50051` is lexicographically smallest, so it becomes the primary. Both proxies accept the same `--cluster` flag and apply the same rule:

```
endpoints = sorted(e.lower().strip() for e in cluster_arg.split(","))
primary   = endpoints[0]
replicas  = endpoints[1:]
```

When you're developing and debugging on one machine (like your PC), you can simply use `localhost` for a hostname:

```
python3 server.py --listen localhost:12345 \
                  --cluster localhost:12345
```

Note that you're specifying a TCP port number explicitly (12345 in the above example). If another service is using it, your attempt to bind to it will fail and you'll need to pick another number.

Write Path

All write operations (`Put`, `Delete`, `Update`, `Reset`) are handled exclusively by the primary.

1. The client sends the write RPC to the primary.
2. The primary validates the request and applies it locally.
3. The primary constructs a `WriteOp` and sends it via `ApplyWrite` to each replica. If the replica list is empty (single-node mode), this step is skipped.
4. The primary counts acknowledgments, including its own local apply. It returns `OK` to the client once it has received acknowledgments from a **majority** of the cluster. For a 3-node cluster, the majority is 2 (which means the primary must get at least one reply). For a single-node cluster, the majority is 1, which is satisfied by the local apply.
5. If a replica is unreachable, the primary logs the failure. The write still commits as long as a majority acknowledges.
6. If the primary cannot reach a majority, it does not commit and returns `UNAVAILABLE` to the client.

Replicas **must reject** any client call to `Put`, `Delete`, `Update`, or `Reset` with status `FAILED_PRECONDITION` and an error message directing the client to the primary. Replicas accept only `Get`, `List`, `Stats`, and `ApplyWrite`.

Read Path

Read operations (`Get`, `List`, `Stats`) may be sent to any node. The default is to contact the primary, but round-robin across all nodes is also acceptable. Document your choice in your write-up.

Failure Semantics

The failure model is intentionally simple. Here is the complete, authoritative list of what is and is not required.

What is required:

- A write returns `OK` only after a majority of nodes have acknowledged it. If the primary cannot reach a majority, it returns `UNAVAILABLE` and the write is not applied.
- If the primary is unreachable, write RPCs return `UNAVAILABLE`.
- Replicas that are down do not block a write from committing, as long as a majority still acknowledges.

What is explicitly not required:

- No failover. If the primary goes down, writes are unavailable until it is restarted manually. There is no promotion of a replica to primary.
- No leader election. The primary is fixed at startup by the lexicographic rule and does not change.
- No restart recovery. When any node restarts, its in-memory state is gone. It does not re-sync from surviving nodes.
- No rollback. If the primary applies a write locally and then fails to reach a majority, you do not need to undo the local apply. This is an edge case that would require significant complexity to handle correctly, and it is not graded.

These omissions are deliberate. They keep the assignment focused on the core mechanics of majority-commit replication without requiring you to implement a full consensus protocol.

REST API

The REST proxy exposes the following endpoints. All writes are forwarded to the primary; reads may go to any node.

HTTP Method	Path	Maps To
`PUT /objects/{key}`	Body: raw bytes	`Put`
`GET /objects/{key}`		`Get`
`DELETE /objects/{key}`		`Delete`
`PATCH /objects/{key}`	Body: raw bytes	`Update`
`GET /objects`		`List`
`DELETE /objects`		`Reset`
`GET /stats`		`Stats`

HTTP status codes must map from gRPC status: `200 OK`, `404 Not Found`, `409 Conflict` (ALREADY_EXISTS), `400 Bad Request` (INVALID_ARGUMENT), `503 Service Unavailable` (UNAVAILABLE), `412 Precondition Failed` (FAILED_PRECONDITION).

For `Get`, return the raw bytes as the response body with `Content-Type: application/octet-stream`. For `List` and `Stats`, return JSON. Do not base64-encode or otherwise transform values.

The REST proxy accepts `--cluster` and `--port` flags.

CLI Commands

```
put <key> <value>
get <key>
delete <key>
update <key> <value>
list
reset
stats
```

The `<value>` in `put` and `update` may be an unquoted token or a quoted string. The CLI does not need to support binary data.

The output must clearly indicate success or error status for each command and be consistent enough for a grading script to parse.

The CLI accepts a `--cluster` flag.

Concurrent Correctness

Your server must be safe for concurrent clients. Include a concurrent correctness test (`test/test_concurrent.py`) in your submission. The test must:

1. Launch 20 threads within a single process, each performing 200 mixed `put`/`get`/`delete` operations on overlapping keys simultaneously.
2. After all threads complete, assert that the final state is consistent: no phantom keys, no corrupted values, and `stats` counters match the number of successful operations recorded by the threads.

Report what synchronization primitives you used and why you chose them over alternatives.

Conformance Testing

We will run a conformance test suite against your server. The suite exercises every operation including all error paths:

- `get` and `delete` on a nonexistent key.
- `put` on a key that already exists.
- `update` on a nonexistent key.
- `put` with a key that violates key rules.
- `put` with a value exceeding 1 MiB.
- `list` after a mix of puts and deletes.
- `stats` counter accuracy across a known sequence of operations.
- `reset` clears all objects and all counters.
- Write rejection on a replica: client sends `Put` directly to a replica and receives `FAILED_PRECONDITION`.
- Majority commit: one replica is killed; writes must still succeed.

The test harness starts a 3-node cluster using the `--listen` and `--cluster` convention above. Your server must be ready to accept connections within ten seconds of process start.

Benchmarks

Run all benchmarks on iLab machines with the server cluster on different machines from the client processes. Report results as labeled tables or plots. Include one to two pages of analysis in your write-up.

“N concurrent clients” means N separate Python processes running simultaneously. Use a benchmark launcher script to start them.

About p values
A percentile latency tells you how long operations take at a given point in the distribution. If you collect 10,000 latency measurements and sort them, the p50 (median) is the value at position

5,000: half of the requests were faster, half were slower. The p95 is the value at position 9,500: 95% of requests were faster than this. The p99 is the value at position 9,900. p99 is important in practice because in a system handling thousands of requests per second, the slowest 1% is not rare: it affects dozens of requests every second. p50 tells you about the typical case, while p95 and p99 tell you about the tail.

Benchmark 1: Throughput vs. Concurrency

Fix the object size at 4 KiB. Run a sustained `put` workload for 30 seconds at each concurrency level: 1, 2, 4, 8, 16, and 32 client processes. Report operations per second at each level. Repeat for `get`. Plot both curves on the same axes.

Explain the shape of the curves. Where does the throughput plateau and why? Does the read curve differ from the write curve, and what does that tell you about the bottleneck?

Benchmark 2: Replication Cost

Run a sustained `put` workload at 8 concurrent clients with 4 KiB objects for 30 seconds under three configurations: a single standalone server (no replication), a 2-node cluster (primary plus one replica, majority = 2), and a 3-node cluster (primary plus two replicas, majority = 2). Report throughput and p99 latency for each configuration as a table.

Explain what the numbers tell you about the cost of synchronous replication and majority-wait. Does throughput scale linearly with the number of nodes? Why or why not? What does the p99 latency trend suggest about where the bottleneck is in a replicated write?

Deliverables

Submit a single zip or tar archive containing:

1. `server.py`: the object store server.
2. `restproxy.py`: the REST proxy (skeleton filled in).
3. `cli.py`: the CLI proxy.
4. `objectstore.proto`: the proto file we supplied, unmodified.
5. `objectstore_pb2.py` and `objectstore_pb2_grpc.py`: generated stubs.
6. `requirements.txt`: identify all Python dependencies & versions.
7. `test/` : your concurrent correctness test and any additional tests.
8. `bench/` : scripts for both benchmarks.
9. `report.pdf` (two to three pages) covering:
   - Benchmark results with analysis for both benchmarks.
   - A short design and validation note (roughly one page or less) with three parts:
     1. One design choice you made and why; for example, your locking strategy, how you handle read routing, or how you structured the fan-out. Explain what alternatives you considered and why you chose what you did.
     2. One bug or test failure you encountered during development. Describe what the symptom was and what the root cause turned out to be.
     3. How did you confirm the fix worked?

Grading

Component	Weight
Server correctness (conformance suite)	30%
REST proxy correctness	10%
CLI proxy correctness	10%
Replication and failure semantics	15%
Concurrent correctness test	10%
Benchmark results (two, with scripts)	15%
Write-up quality and analysis	10%

A server that fails basic single-node operations earns at most 50% of the server correctness points regardless of the write-up.

Notes and Hints

The `list` RPC and concurrency. Taking a lock for the entire duration of building the `ListResponse` is fine. Unlike a streaming RPC, you are not holding the lock while sending data over the network. Build the response under the lock, release it, then return the response. This is much simpler than the streaming case.

The REST proxy and binary values. Pass binary values through `PUT /objects/{key}` using `Content-Type: application/octet-stream`. Do not base64-encode values. See `restproxy-guide.md` for details.

Majority commit. For a 3-node cluster, the majority is 2 (the primary plus one replica). Use `concurrent.futures.ThreadPoolExecutor` to fan out `ApplyWrite` calls to replicas in parallel. Count successful acknowledgments, including the primary's own local apply, and commit if that count is >= 2. For a single-node cluster, the replica list is empty, the fan-out does nothing, and the local apply alone satisfies the majority.

Replica write rejection. At the top of each write handler, check whether `--listen` equals the computed primary endpoint. If not, immediately return `FAILED_PRECONDITION` before touching any state.

The `--cluster` normalization rule. Apply `.lower().strip()` to each endpoint before lexicographic comparison to avoid surprising results from mixed case or trailing whitespace.

iLab machine selection. Check that your chosen nodes are lightly loaded before starting a run (the `uptime` command). Record the exact machine names in your write-up so results are reproducible.